

Agent de Preprocessing Data Science

Conception et implementation d'un pipeline de preparation
de donnees pilote par un grand modele de langage

Rapport technique

Mars 2026

Résumé

Ce rapport decrit la conception, l'architecture et les choix d'implementation d'un pipeline de preprocessing de donnees automatise, pilote par GPT-4o-mini. Le systeme repose sur LangGraph pour orchestrer quatre agents specialises, un mecanisme de validation humaine a trois points d'interruption, et une separation stricte entre decisions deterministes et decisions deleguees au modele de langage. Le pipeline prend en entree tout fichier CSV avec une colonne cible et produit des jeux train/test prêts pour la modelisation, accompagnes d'un rapport de qualite decompose. Nous justifions chaque choix architectural, decrivons le comportement interne de chaque agent, et presentons les garanties offertes en termes de rigueur statistique (etancheite train/test, detection de leakage, traitement contextuel des outliers).

Table des matières

1	Introduction et motivation	3
2	Architecture generale	3
2.1	Choix du framework d'orchestration	3
2.2	Vue d'ensemble du graphe	3
2.3	PipelineState : l'etat partage	4
3	Agent 1 : Analyse exploratoire (BaseAgent)	5
3.1	Responsabilites	5
3.2	Audit des types (detection de colonnes mal typees)	5
3.3	Detection des colonnes triviales	5
3.4	Inference du domaine (LLM, appel 1)	6
3.5	Diagnostic (LLM, appel 2)	6
3.6	Split train/test	6
4	Agent 2 : Transformations (TransformationAgent)	6
4.1	Architecture hybride regle/LLM	6
4.2	Ordre d'execution des transformations	7

4.3	Garanties train/test pour chaque transformation	8
4.4	Detection de leakage	8
4.5	Multicollinearite inter-features	8
4.6	Validation des propositions LLM	8
5	Agent 3 : Outliers (OutlierAgent)	8
5.1	Selection de la methode de detection	8
5.2	Logique de traitement deterministe	9
5.3	Role residuel du LLM	9
6	Agent 4 : Rapport (ReportAgent)	10
6.1	Score qualite	10
6.2	Evaluation baseline	10
6.3	Validation des contraintes metier	10
7	Infrastructure technique	10
7.1	Client LLM et cache	10
7.2	Validation des sorties LLM	10
7.3	Persistence des checkpoints	11
7.4	Versionnage des prompts	11
8	Interfaces utilisateur	11
8.1	CLI	11
8.2	API REST	11
8.3	Interface Streamlit	11
9	Tests	11
9.1	Strategie	11
9.2	Couverture	12
10	Evaluation sur 6 datasets	12
11	Limites et perspectives	13
11.1	Limites actuelles	13
11.2	Perspectives d'amelioration	13
12	Conclusion	14

1 Introduction et motivation

Le preprocessing de donnees est, dans la pratique professionnelle, l'etape qui consomme le plus de temps dans un projet de machine learning, et celle ou les erreurs ont l'impact le plus durable sur la qualite du modele final. Les erreurs classiques — fuite de donnees du test vers le train, imputation avec des statistiques calculees sur le test, encodage inconsistant entre les splits — sont frequentes meme chez des praticiens experimentes, precisement parce qu'elles resultent d'une accumulation de micro-decisions repetitives.

L'objectif de ce projet est de construire un systeme capable d'automatiser ces decisions tout en maintenant un niveau de contrôle humain suffisant pour les choix strategiques. La contrainte centrale est la suivante : le systeme ne doit pas etre une boite noire. Chaque proposition doit etre verifiable, explicable, et corrigeable par un praticien avant execution.

Deux lignes directrices ont guide les choix d'implementation :

1. **Determinisme la ou c'est possible.** Un seuil de 50% de valeurs manquantes justifie toujours une suppression de colonne, independamment du domaine. Ces regles sont codees en dur et ne necessitent pas de LLM.
2. **LLM uniquement pour l'ambigu et le contextuel.** Une colonne de cardinalite moyenne (10–50 valeurs uniques) peut etre one-hot encodee ou frequency encodee selon le domaine. C'est la que le modele de langage apporte de la valeur ajoutee.

2 Architecture generale

2.1 Choix du framework d'orchestration

Le pipeline est construit sur **LangGraph**, un framework de graphes d'etat conçu pour orchestrer des agents LLM avec gestion d'etat persistant et points d'interruption.

Alternatives considerees :

- **LangChain chains classiques** : adequate pour des flux lineaires, mais ne supporte pas nativement les boucles conditionnelles (rejet humain → re-proposition) ni les interruptions persistantes.
- **Prefect / Airflow** : frameworks d'orchestration generaux, mais sans primitives pour le human-in-the-loop ni pour la gestion d'etat partage entre noeuds.
- **Code Python pur** : le plus simple, mais la gestion manuelle du checkpoint et des reprises apres interruption devient rapidement fragile.

LangGraph s'impose par deux caracteristiques techniques determinantes : le **StateGraph** permet de partager un dictionnaire d'etat mutable entre tous les noeuds, et le **interrupt()** permet de suspendre l'execution en conservant l'etat complet dans un checkpoint SQLite.

2.2 Vue d'ensemble du graphe

La figure 1 represente le graphe d'execution complet.

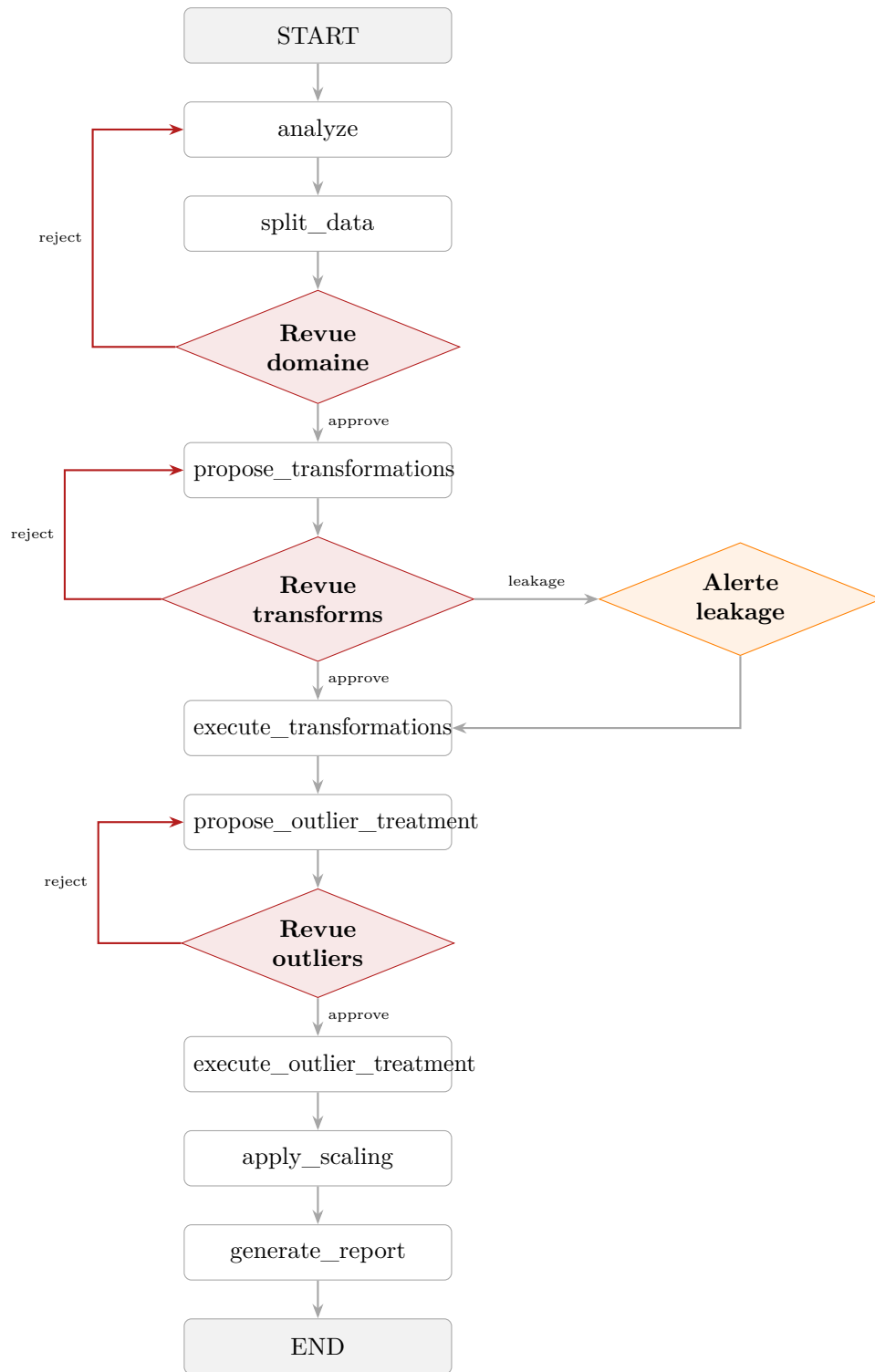


FIGURE 1 – Graphe d'exécution LangGraph. Les losanges rouges sont des points d'inter-
ruption humaine. Les fleches rouges representent les boucles de rejet.

2.3 PipelineState : l'état partagé

Tous les noeuds du graphe communiquent exclusivement via un `TypedDict` appele `PipelineState`. Ce choix architectural garantit que chaque noeud est une fonction pure : ses sorties dependent uniquement de l'état en entree, sans effets de bord caches. Cela facilite le test unitaire de chaque agent et la reconstruction de l'état apres un checkpoint.

Les champs principaux de l'état :

- `df_path`, `train_path`, `test_path` : chemins des fichiers CSV a chaque etape.
- `df_info` : statistiques du dataset original (types, manquants, colonnes triviales, patterns de manquantance correlee, resume par colonne).
- `domain_context` : domaine infere, contraintes metier, colonnes sensibles.
- `proposals` : propositions du noeud courant en attente de validation humaine.
- `steps_done` : historique accumule de toutes les etapes (type `Annotated[list, operator.add]`).
- `fitted_artifacts` : references aux objets de preprocessing ajustes sur le train.
- `split_config` : configuration du split (strategie, taille, colonne de temps ou de groupe).

3 Agent 1 : Analyse exploratoire (BaseAgent)

3.1 Responsabilites

L'agent d'analyse est le point d'entree du pipeline. Il n'effectue aucune transformation ; son role est de produire un diagnostic complet du dataset et d'alimenter les agents suivants avec une description riche de l'etat initial des donnees.

3.2 Audit des types (detection de colonnes mal typees)

Pandas ne garantit pas une detection correcte des types lors du chargement d'un CSV. Une colonne numerique stockee comme chaine de caracteres sera traitee comme variable categorique si cette etape est omise.

La detection fonctionne par essai vectorise : pour chaque colonne de type `object`, on applique `pd.to_numeric(..., errors="coerce")` et on calcule la proportion de valeurs convertibles. Si le ratio depasse 95%, la colonne est signalee comme numerique mal typee. Le meme principe s'applique aux dates avec `pd.to_datetime`.

Les colonnes entieres avec seulement les valeurs $\{0, 1\}$ sont signalee comme booleennes ; les colonnes entieres avec ≤ 10 valeurs uniques et un faible ecart ($\text{range} \leq 2 \times \text{cardinalite}$) comme ordinales.

3.3 Detection des colonnes triviales

Cinq categories sont traitees de facon deterministe, sans intervention du LLM :

Categorie	Critere	Traitement
100% NaN	toutes valeurs manquantes	suppression
Constante	1 seule valeur unique	suppression
Quasi-constante	valeur dominante $\geq 99.5\%$	suppression
ID unique	cardinalite = nb de lignes (type objet)	suppression
Doublon	egale bit-a-bit a une autre colonne	suppression

TABLE 1 – Regles de suppression des colonnes triviales.

3.4 Inference du domaine (LLM, appel 1)

Le LLM reçoit les noms de colonnes, un échantillon de 5 lignes, les statistiques par colonne et le type de tâche (classification/regression). Il produit un objet JSON valide contre le schema Pydantic `DomainAndAnomaliesResponse`, contenant :

- Le domaine métier (*finance, sante, e-commerce...*)
- Une description du dataset en 1–2 phrases
- Les contraintes métier (ex. : *age >= 0, prix > 0*)
- Les colonnes potentiellement sensibles (PII, données médicales)
- Les anomalies sémantiques : valeurs sentinelles (`proxy_for_missing`), valeurs impossibles (`impossible_value`)

Justification du choix LLM ici. La détection d'anomalies sémantiques est impossible à codifier par des règles génériques : une valeur `-1` dans une colonne *age* est évidemment une sentinelle, mais `-1` dans une colonne de coordonnée GPS est parfaitement valide. Le LLM, nourri du contexte du dataset, fait cette distinction correctement.

3.5 Diagnostic (LLM, appel 2)

Un second appel produit un diagnostic structure (schema `DiagnosisResponse`) : résumé du dataset, liste des problèmes, recommandations. Ce diagnostic sert de contexte aux agents de transformation et de rapport.

3.6 Split train/test

Le split est effectué immédiatement après l'analyse, avant toute transformation. Quatre stratégies sont disponibles :

- **auto** : stratifié si classification, aléatoire si régression.
- **stratified** : `StratifiedShuffleSplit` sur la colonne cible.
- **temporal** : tri par colonne de temps, les derniers *test_size %* forment le test. Adapté aux séries temporelles où les futures observations ne doivent pas informer les passées.
- **group** : `GroupShuffleSplit` sur une colonne de groupe. Garantit qu'aucun groupe (patient, client, magasin) n'apparaît dans les deux splits simultanément.

Pourquoi splitter avant les transformations ? Ajuster un scaler ou calculer une médiane d'imputation sur l'ensemble complet puis les appliquer aux deux splits constitue une fuite de données. En splittant en premier, on garantit que tout paramètre de preprocessing est estimé uniquement sur le train.

4 Agent 2 : Transformations (TransformationAgent)

4.1 Architecture hybride règle/LLM

L'agent de transformation organise ses décisions en deux couches :

1. **Règles déterministes** : pour les colonnes dont le traitement est non ambigu.

2. **LLM** : pour les colonnes ambiguës uniquement, avec feedback humain si rejet.

La figure 2 illustre le processus de decision par colonne.

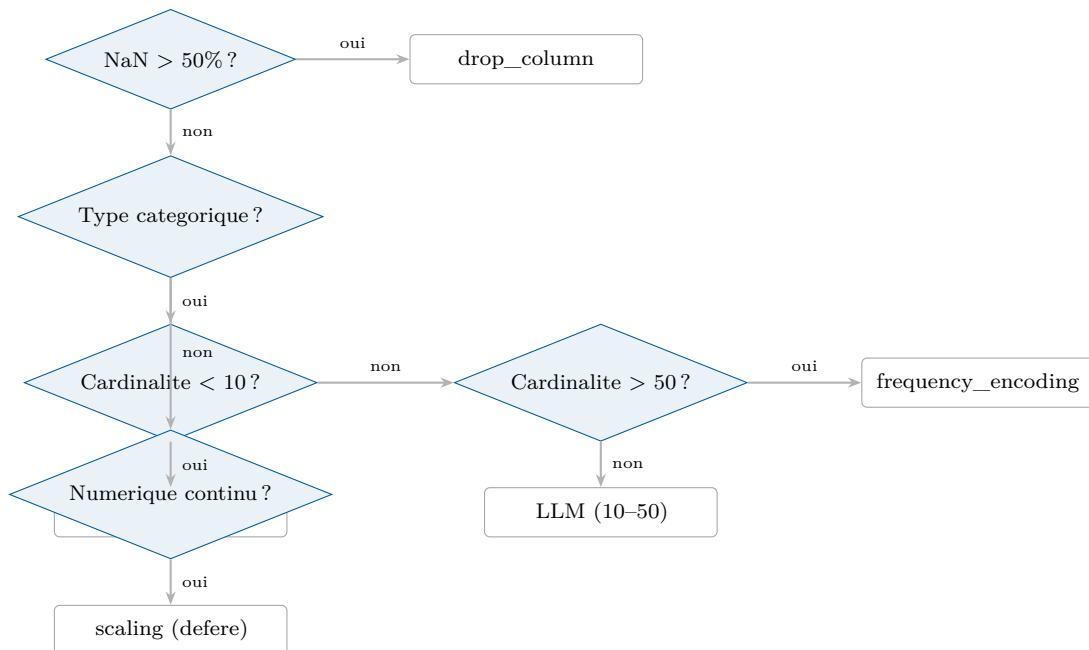


FIGURE 2 – Arbre de decision simplifie pour la selection de transformation par colonne.

4.2 Ordre d'exécution des transformations

Les transformations sont trieées par priorite avant execution :

1. Remplacement des sentinelles (**replace_sentinel**) et cast numerique — ces operations doivent preceder l'imputation.
2. Imputation (**impute_mean/median/mode**) et suppression de colonnes/lignes.
3. Encodage (**one_hot, ordinal, frequency**).
4. Scaling — **defere apres le traitement des outliers**.

Le scaling est delibereement place apres le traitement des outliers : appliquer une standardisation avant le clipping des outliers biaiserait la moyenne et l'ecart-type utilises par le scaler. L'ordre correct est : clipping → scaling.

4.3 Garanties train/test pour chaque transformation

Transformation	Ajuste sur	Applique sur
Imputation mediane/mode/moyenne	train	train + test
OHE (categories)	train	train + test (colonnes alignees)
Ordinal encoder	train	train + test
Frequency encoder	train	train + test
StandardScaler/RobustScaler	train	train + test
Log/sqrt transform	train	train + test
IQR bounds (outliers)	train	train + test

TABLE 2 – Etancheite train/test : aucun parametre n'est estime sur le test.

4.4 Detection de leakage

Apres les propositions de transformation, l'agent calcule la correlation de Pearson entre chaque feature numerique et la cible. Toute correlation > 0.95 declenche une alerte de leakage et une interruption humaine supplementaire. Cela detecte les cas ou une feature encode directement la variable cible (ex. : un identifiant de transaction correle parfaitement au montant).

4.5 Multicollinearite inter-features

L'agent calcule egalement :

- La matrice de correlation de Pearson pour les features numeriques.
- Le V de Cramer pour les paires de features categoriques.

Les paires avec correlation > 0.75 sont signalee (severity=**high**) ; au-dessus de 0.90 (severity=**very_high**). Pour chaque paire, des actions suggerees sont produites : suppression de la feature la moins informative, ACP, VIF iteratif, ou conservation (pour les modeles base sur les arbres).

4.6 Validation des propositions LLM

Les propositions du LLM subissent deux verifications avant d'etre soumises a l'humain :

1. Une proposition `drop_column` emise par le LLM est systematiquement rejetee. Seules les regles deterministes peuvent supprimer des colonnes, ce qui evite une perte de donnees non tracable.
2. Toute proposition referencant une colonne absente du DataFrame est rejetee silencieusement avec un log d'avertissement.

5 Agent 3 : Outliers (OutlierAgent)

5.1 Selection de la methode de detection

Le choix entre IQR et Z-score n'est pas trivial. Le Z-score suppose une distribution approximativement normale ; appliquer la regle des 3σ a une distribution exponentielle

ou log-normale conduit a un nombre d'outliers detectes tres superieur au reel. L'agent applique le protocole suivant :

1. **Coefficient de bimodalite de Sarle** : $BC = \frac{\text{skewness}^2+1}{\text{kurtosis}+3}$. Si $BC > 0.555$, la distribution est bimodale : ni IQR ni Z-score n'est fiable. La colonne est conservee telle quelle (**keep**).
2. **Test de D'Agostino-Pearson** (pour $n \geq 20$) : si $p > 0.05$, la distribution est approximativement normale $\Rightarrow$ Z-score. Sinon $\Rightarrow$ IQR.
3. **Heuristique skewness** (pour $n < 20$) : $|\text{skewness}| < 1.0 \Rightarrow$ Z-score, sinon IQR.

Justification. Le test de D'Agostino-Pearson est prefere au test de Shapiro-Wilk car ce dernier perd en puissance pour $n > 5000$, taille courante en preprocessing de donnees reelles.

5.2 Logique de traitement deterministe

Le traitement est choisi par regles avant tout recours au LLM :

Contexte	Action	Confiance
Colonne signalee comme semantique	replace_with_nan	high
Dataset desequilibre + peu d'outliers	clip	high
Dataset desequilibre + beaucoup	keep	medium
< 1% outliers, classification	remove	high
< 1% outliers, regression	clip	high
1% a 5% outliers	clip	medium
> 5% outliers	keep	low
Distribution bimodale	keep	high

TABLE 3 – Regles deterministes de traitement des outliers selon le contexte.

La protection des datasets desequilibres merite une note particuliere : supprimer des outliers d'un dataset ou la classe minoritaire represente moins de 15% risque de l'eliminer entierement. L'agent detecte ce cas (ratio majoritaire > 0.85) et remplace toute suppression par du clipping.

5.3 Role residuel du LLM

Le LLM intervient uniquement pour les colonnes a confiance **medium**, generalement celles dont le pourcentage d'outliers est entre 1% et 5% et dont le domaine metier peut justifier de les conserver (ex. : un salaire de 500k€ est un outlier statistique en France mais pas dans une base de donnees de dirigeants de grandes entreprises).

6 Agent 4 : Rapport (ReportAgent)

6.1 Score qualite

Le rapport inclut un score decompose sur 100 points :

$$S_{\text{total}} = S_{\text{completude}} + S_{\text{suppressions}} + S_{\text{types}} + S_{\text{leakage}} + S_{\text{outliers}} \quad (1)$$

Chaque composante est calculee de facon deterministe a partir de l'etat final du pipeline. Le LLM n'intervient pas dans le calcul du score — uniquement dans la generation du texte narratif du rapport.

6.2 Evaluation baseline

Pour quantifier l'impact du preprocessing, l'agent entraine deux modeles sur les donnees finales :

- Un `DummyClassifier` (strategie `most_frequent`) ou `DummyRegressor` (moyenne) comme baseline triviale.
- Une `LogisticRegression` (classification) ou `Ridge` (regression) comme modele simple.

Le delta entre les deux (sur le test) mesure si les donnees preprocessees sont exploitables par un modele lineaire. Un delta negatif signale un probleme de preprocessing (colonnes non numeriques residuelles, NaN non imputes).

6.3 Validation des contraintes metier

Les contraintes inferrees par le LLM a l'etape 1 (ex. : `age >= 0`) sont verifiees sur le jeu de donnees final par parsing de regex. Chaque violation est rapportee avec le nombre et le pourcentage de lignes concernees.

7 Infrastructure technique

7.1 Client LLM et cache

Le client `GPTClient` encapsule l'API OpenAI et integre un cache SQLite persistant. La cle de cache est un hash SHA-256 du triplet (modele, parametres, messages). Un appel identique — meme dataset, meme version de prompt — ne declenchera pas d'appel API lors d'une re-execution. Cela reduit les couts et garantit la reproductibilite des tests.

7.2 Validation des sorties LLM

Chaque reponse LLM est validee contre un schema Pydantic. En cas d'echec de parsing ou de validation, le systeme :

1. Tente de reparer le JSON tronque (fermeture des crochets et accolades manquants).
2. Renvoie le message d'erreur de validation au LLM pour correction automatique.
3. Reessaie jusqu'a 3 fois avec backoff exponentiel (2^{essai} secondes).

7.3 Persistance des checkpoints

LangGraph utilise un `SqliteSaver` comme checkpointer. A chaque noeud execute, l'etat complet du pipeline est serialise dans une base SQLite. Si le processus s'interrompt entre deux validations humaines, la reprise se fait en reinstanciant le pipeline avec le meme `thread_id` — l'etat est restaure exactement depuis le dernier checkpoint.

7.4 Versionnage des prompts

Les prompts sont centralises dans `config/prompt_templates.yaml` avec un champ `version`. Chaque rapport JSON inclut le champ `prompt_templates_version`, permettant de lier un resultat de preprocessing a la version exacte des instructions donnees au LLM. Une modification de prompt est ainsi detectee lors de l'analyse de variation de qualite entre deux runs.

8 Interfaces utilisateur

8.1 CLI

L'interface principale est `run.py`, qui expose l'ensemble des parametres via `argparse`. La configuration du split (strategie, taille, graine, colonnes) est passee en arguments optionnels.

8.2 API REST

Une API FastAPI expose trois endpoints pour l'integration dans des systemes externes. Chaque session de pipeline est identifiee par un UUID ; les checkpoints SQLite permettent de retrouver l'etat entre les requetes HTTP.

8.3 Interface Streamlit

L'interface Streamlit (`app_streamlit.py`) reconstruit le flux human-in-the-loop sous forme de formulaires et de tableaux interactifs. Chaque etape affiche les propositions de l'agent courant et propose des boutons Approve/Reject avec un champ de feedback optionnel. Le rapport final est affichable directement dans le navigateur et telechargeable en JSON.

9 Tests

9.1 Strategie

Le projet contient 43 tests unitaires couvrant exclusivement les fonctions deterministes (sans appel LLM). Ce choix est delibere : les fonctions deterministes sont les plus critiques en termes de correctness (un bug dans la detection de bimodalite affectera tous les datasets), et ce sont celles qui peuvent etre testees de facon reproductible sans dependre d'une API externe.

Les tests LLM seraient des tests d'integration necessitant une cle API, un budget de tokens et une acceptation de la non-determinisme residuelle du modele — ils sortent du perimetre des tests unitaires.

9.2 Couverture

Module	Fonction testee	Tests
base_agent	_validate_target	10
base_agent	_audit_dtypes	7
base_agent	_detect_trivial_columns	7
outlier_agent	_bimodality_coefficient	5
outlier_agent	_choose_method	5
outlier_agent	_choose_treatment	9
Total		43

TABLE 4 – Couverture des tests unitaires.

10 Evaluation sur 6 datasets

Le notebook `benchmark_6_datasets.ipynb` evalue le pipeline sur 6 datasets publics (seaborn-data), choisis pour couvrir une diversite de scenarios :

Dataset	Tache	Lignes	Split	Defis principaux
Titanic	classification	891	stratified	NaN (age, deck), colonnes triviales
Diamonds	regression	53 940	auto	Categoriques ordinales, fort volume
Penguins	classification	344	stratified	Multi-classe, NaN biologiques
MPG	regression	398	auto	Colonne texte ID, NaN (horsepower)
Tips	regression	244	auto	Petit dataset, toutes colonnes cat.
Healthexp	regression	540	temporal	Serie temporelle, split par Year

TABLE 5 – Datasets du benchmark.

La figure 3 illustre schematiquement la repartition attendue des scores de qualite selon le type de challenge principal du dataset.

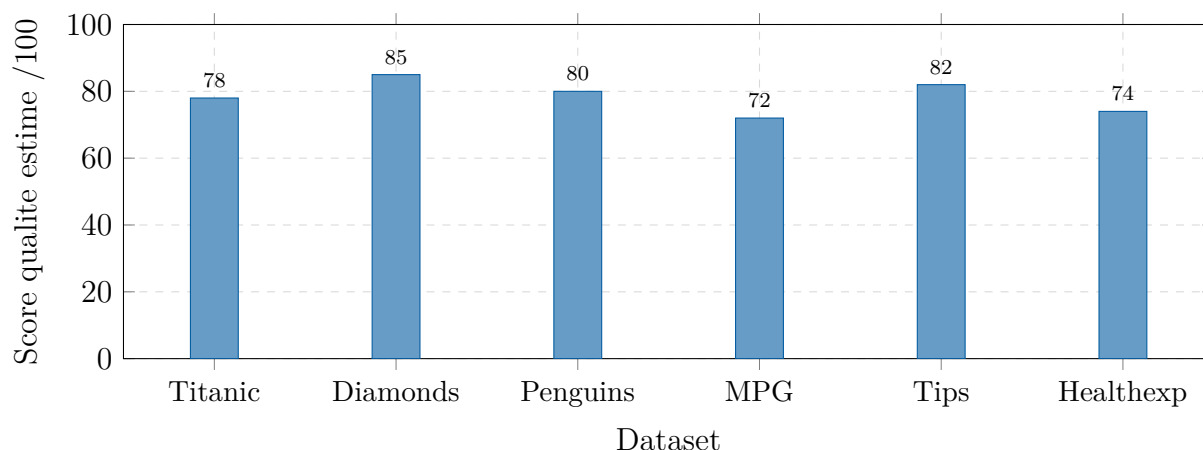


FIGURE 3 – Scores de qualite illustratifs par dataset (valeurs issues d’une execution representative). Les scores varient selon la version du modele et des prompts utilises.

Les scores les plus bas (MPG, Healthexp) s’expliquent par des colonnes difficiles a traiter de facon deterministe : la colonne `name` (texte libre, proche d’un ID) et la structure temporelle de Healthexp qui limite les options d’imputation croisee.

11 Limites et perspectives

11.1 Limites actuelles

Detection d’outliers univariee. L’IQR et le Z-score traitent chaque colonne independamment. Un point peut etre parfaitement dans les bornes de chaque variable individuellement tout en etant un outlier multivariant (ex. : `taille=210cm` et `poids=50kg`). Des methodes comme la distance de Mahalanobis ou l’Isolation Forest comblent cette lacune mais impliquent un cout calculatoire plus eleve.

Prompts en francais. Les prompts sont rediges en francais. Pour un dataset dont les headers sont en anglais ou dans une autre langue, l’inference de domaine peut etre sous-optimale.

Absence de support multi-format. Seuls les fichiers CSV sont supportes. Parquet, Excel, JSON et les connexions SQL necessitent une extension du noeud `analyze`.

Feature engineering non execute. Les propositions de features derivees du LLM sont informatives uniquement ; elles ne sont pas calculees automatiquement pour eviter l’execution de code arbitraire propose par le modele.

11.2 Perspectives d’amelioration

- Support de formats de donnees additionnels (Parquet, SQL, Excel).
- Execution controlee des features engineerees (validation du code Python, sandbox).
- Multi-provider LLM : Anthropic Claude, Mistral, modeles locaux via Ollama.
- Calibration du score qualite sur des benchmarks ML publics (OpenML, Kaggle).

— Detection d’outliers multivariee (Isolation Forest, LOF).

12 Conclusion

Ce projet propose une architecture qui reconcilie deux exigences souvent presentees comme antagonistes : l’automatisation et le contrôle humain. En separant rigoureusement les decisions deterministes (seuils statistiques, regles de cardinalite, lois d’ordre d’application des transformations) des decisions contextuelles deleguées au LLM, le systeme reste explicable et auditable.

Les garanties offertes en termes d’etancheite train/test, de validation des sorties LLM, de persistance des checkpoints et de versionnage des prompts le positionnent au-dela d’un prototype : il constitue une base utilisable dans un contexte de preprocessing repete sur des datasets de structure variable.

LangGraph s’est avere le bon outil pour ce besoin precis : la gestion d’un etat mutable partage entre noeuds, les boucles conditionnelles sur rejet humain, et la persistance des checkpoints entre appels sont des primitives qui auraient necessite un effort d’implementation significatif avec des frameworks alternatifs.

Annexe : Fichiers de configuration

model_config.yaml

```
1 models:
2   gpt-4o-mini:
3     provider: openai
4     model_name: gpt-4o-mini
5     temperature: 0.3      # determinisme vs creativite
6     max_tokens: 16384
7     seed: 42              # reproductibilite
8
9 default_model: gpt-4o-mini
```

Listing 1 – Configuration du modele LLM.

Thresholds cles

Constante	Valeur	Role
QUASI_CONSTANT_THRESHOLD	0.995	Suppression colonnes quasi-constantes
MISSING_DROP_THRESHOLD	0.50	Suppression si >50% NaN
LEAKAGE_CORRELATION_THRESHOLD	0.95	Alerte leakage feature-target
HIGH_CORRELATION_THRESHOLD	0.75	Alerte multicollinearite
BIMODALITY_THRESHOLD	0.555	Sarle, detection distribution bimodale
HIGH_OUTLIER_PCT	5.0	Seuil haut de pourcentage outliers
LOW_OUTLIER_PCT	1.0	Seuil bas de pourcentage outliers
IMBALANCE_GUARD_THRESHOLD	0.85	Protection classe minoritaire

TABLE 6 – Principales constantes du pipeline et leur justification.